Asignatura Programación 1

Profesorado en Informática

Docente a cargo:
Prof. Brassesco María Virginia

Carga Horaria:
128 horas

Presentación de Programa:
Febrero 2025

Firma Virginia Brassesco

a. Fundamentación **1**
b. Objetivos o propósitos **3**
Objetivos pedagógicos 4
c. Contenidos **4**
Unidad 1: Introducción a la programación y resolución de problemas 4
Unidad 2: Fundamentos de programación imperativa 5
Unidad 3: Estructuras de control 5
Unidad 4: Funciones y modularización 6
Unidad 5: Prácticas de programación 6
Unidad 6: Introducción a estructuras de datos básicas 7
Contenido Opcional - Unidad 7: Temas avanzados de programación 7
Tema 1: Tipos de datos computacionales de dos dimensiones (matrices y diccionarios) 7
Tema 2: Interpretación de documentación de módulos y funciones de terceros 8
Tema 3: Definición de tests de caja negra (Partición de Equivalencia) 8
Tema 4: Definición de tests de caja blanca (Cobertura de ramas, condiciones, funciones) 9
d. Metodología y evaluación **9**
Modalidad de cursada 9
Seguimiento del avance 10
Herramientas y buenas prácticas 10
Evaluación y calificación 11
e. Requisitos para la acreditación **11**
f. Bibliografía Obligatoria **11**
g. Bibliografía Complementaria **11**

a. Fundamentación

Esta materia es la primera del área Programación que deben cursar las y los estudiantes y, por lo tanto, presenta nociones fundamentales de la disciplina sin suponer conocimientos previos en la temática. Por este motivo se concentra en algunos aspectos técnicos basales. También se enfatizan fuertemente algunos conceptos centrales que atraviesan toda la disciplina, como la idea de algoritmo o estrategia de resolución de problemas. Si bien se trata de un curso introductorio, los contenidos y habilidades que se trabajan servirán de bases para las siguientes materias, tanto en esta área como en otras, donde se continuará, diversificará y profundizará su desarrollo.

El alcance de este curso abarca algunas herramientas centrales de la programación imperativa que están presentes en la mayoría de los lenguajes y que, además de ser accesibles para un/a programador/a principiante, se utilizan muy frecuentemente. También se incluyen generalidades del funcionamiento de los lenguajes de programación, para comprender el recorrido desde la separación del problema original en partes más pequeñas, la escritura del algoritmo, la escritura del código y finalmente su ejecución.

En cuanto al enfoque didáctico, las y los estudiantes comienzan a programar utilizando bloques gráficos para poder prescindir de la enseñanza de la sintaxis rígida mientras se familiarizan con los primeros conceptos de programación imperativa, intercalando además actividades sin computadoras. Usando de base los conceptos trabajados en la programación en bloques, se intercalan actividades sobre un lenguaje textual, siendo acompañadas/os en la transición. Todo este trayecto se organiza alrededor de la práctica. Se propone un contexto en que las competencias técnicas (propiciadas a partir de la resolución de ejercicios o desafíos de programación) se interrelacionan con las conceptualizaciones generales (suscitadas por la reflexión conjunta sobre el proceso de elaboración de soluciones y su análisis).

Uno de los conceptos generales más importantes que se abordan en esta instancia es el de la legibilidad, característica de los programas que los hace más fáciles de leer y comprender por una persona. Esta propiedad cae dentro de la categoría de buenas prácticas de programación, dado que resulta imprescindible para lograr programas claros y fáciles de entender, permitiendo el trabajo colaborativo y el intercambio de ideas y soluciones. Este intercambio se da bajo la idea de "programa como documento de trabajo", es decir, entender al texto de un programa como un elemento sobre el que se plasma una propuesta que, al volverse visible, habilita la comunicación y la discusión de ideas.

También se considera un eje central pues ordena el proceso de elaboración, dirigiendo un análisis profundo, tanto del problema como de la solución en construcción. Se trabaja bajo la premisa de que un programa legible es el resultado final de plantear una estrategia para un problema, identificar sus partes más relevantes y colocarles nombres adecuados que reflejen, de la manera más fiel posible, en qué consiste cada una. Este proceso, además de favorecer la claridad de los programas, alienta el razonamiento en un nivel de abstracción más alto con el objetivo de facilitar la elaboración de soluciones a partir de una comprensión

más acabada del problema, la solución planteada y la propia práctica.

b. Objetivos o propósitos

Que las y los estudiantes sean capaces de:

1. **Razonar a distintos niveles de abstracción sobre los problemas y sus soluciones:**
    - **Nivel de problema:** Visualizar la solución del problema como una caja negra, donde el enfoque es identificar qué tipo de algoritmo es adecuado para resolverlo.
    - **Nivel de algoritmo:** Operar con el algoritmo que soluciona el problema como un objeto abstracto, independiente de un lenguaje de programación específico. Usualmente pseudocódigo.
    - **Nivel de programa:** El algoritmo propuesto escrito en un lenguaje de programación concreto ejecutable con posibles parámetros de entrada.
    - **Nivel de ejecución:** Una instancia de ejecución específica con argumentos definidos, en una máquina específica.
2. **Entender la noción de algoritmo como una estrategia para resolver un problema:**
    - Reconocer que un algoritmo es una secuencia de pasos bien definidos que transforman una entrada en una salida, y que puede ser implementado a distintos niveles de abstracción.
3. **Expresar soluciones a situaciones computacionales en términos de algoritmos:**
    - Desarrollar la capacidad de traducir problemas del mundo real en algoritmos, considerando los distintos niveles de abstracción y las características del problema.
4. **Identificar subproblemas relevantes dentro de un problema mayor y expresar una estrategia para resolver este último en términos de los primeros:**
    - Aprender a descomponer problemas complejos en subproblemas más manejables, y a construir soluciones modulares que integren las soluciones de estos subproblemas.
5. **Utilizar un lenguaje de programación para resolver problemas sencillos:**
    - Adquirir habilidades prácticas para implementar algoritmos en un lenguaje de programación, aplicando las herramientas fundamentales de un lenguaje imperativo (variables, estructuras de control, funciones, etc.).
6. **Escribir programas legibles y valorar esta propiedad como una característica positiva de los programas o una buena práctica de programación:**
    - Aprender a escribir código claro, bien estructurado y documentado, reconociendo la importancia de la legibilidad para el mantenimiento y la colaboración en proyectos de software.

7. **Interpretar programas sencillos para predecir su comportamiento sin necesidad de ejecutarlos y detectar errores:**
    - Desarrollar la capacidad de leer y analizar código, entendiendo su lógica y anticipando su comportamiento. Esto incluye la identificación de errores sintácticos y lógicos.
8. **Reconocer qué función cumplen los componentes de un programa e identificar términos para nombrarlos que reflejen esta función:**
    - Familiarizarse con los elementos básicos de un programa (variables, funciones, estructuras de control, etc.) y entender cómo contribuyen a la solución general del problema.
9. **Intercambiar, comprender y discutir sobre programas y algoritmos simples entendidos como soluciones a un problema:**
    - Desarrollar habilidades de comunicación técnica para explicar, analizar y discutir algoritmos y programas, tanto en términos de su implementación como de su diseño conceptual.
10. **Reconocer las correspondencias entre un lenguaje de programación por bloques y uno textual:**
    - Entender las similitudes y diferencias entre lenguajes visuales (por bloques) y textuales, y cómo ambos pueden representar soluciones algorítmicas.
11. **Aplicar técnicas de depuración (debugging) automática y manual y verificación (testing) para detectar y corregir errores en los programas:**
    - Familiarizarse con herramientas y métodos para identificar y solucionar errores en el código, así como para verificar su correcto funcionamiento mediante pruebas automatizadas.
12. **Conocer, de modo general, las características de los compiladores y los intérpretes, y sus diferencias:**
    - Comprender los conceptos básicos de compilación e interpretación, y cómo estos procesos afectan la ejecución de los programas.

Objetivos pedagógicos

- **Fomentar la autonomía**: A través de la autorregulación y el uso de herramientas de verificación, se busca que los estudiantes desarrollen habilidades para **aprender de manera autónoma**.
- **Promover la colaboración**: Mediante trabajos grupales y evaluaciones de pares, se fomenta un ambiente de **aprendizaje colaborativo**.
- **Garantizar la calidad del código**: de forma automática con el uso de linters y casos de tests, y con las devoluciones de docentes y el debate entre pares, se asegura que los estudiantes adopten **buenas prácticas de programación**.

c. Contenidos

Unidad 1: Introducción a la programación y resolución de problemas

- **Temas principales**:
  - Conceptos básicos de programación.
  - Resolución de problemas: algoritmos como estrategia y división en subtareas.
  - Funcionamiento de los lenguajes de programación: compilación e interpretación, código fuente vs. código ejecutable.
- **Objetivos de aprendizaje**:
  - Comprender qué es un algoritmo y cómo se utiliza para resolver problemas.
  - Diferenciar entre compilación e interpretación.
  - Diferenciar entre una implementación en un lenguaje de programación con semántica formal y un algoritmo.
  - Aprender a dividir problemas complejos en subtareas más pequeñas.
- **Referencias**:
  - Downey (2015): Capítulo 1 (**El camino del programa**).
  - Guttag (2016): Capítulo 1 (**Introducción**).
  - **Van Rossum (2015)**: Secciones introductorias del **Tutorial de Python** (para una visión oficial del lenguaje).

Unidad 2: Fundamentos de programación imperativa

- **Temas principales**:
  - Variables: tipos de datos, asignación, acumuladores.
  - Operadores y expresiones lógicas.
  - Entrada y salida de datos.
- **Objetivos de aprendizaje**:
  - Comprender el uso de variables y tipos de datos.
  - Aprender a utilizar operadores aritméticos y lógicos.
  - Implementar programas que interactúen con el usuario mediante entrada y salida de datos.
- **Referencias**:
  - Downey (2015): Capítulo 2 (**Variables, expresiones y sentencias**).
  - Guttag (2016): Capítulo 2 (**Introducción a Python**).
  - **Van Rossum (2015)**: Secciones sobre **Variables y tipos de datos** en el **Tutorial de Python**.

Unidad 3: Estructuras de control

- **Temas principales**:
  - Alternativa condicional (if, else, elif).
  - Repetición: bucles simples (for) y condicionales (while).
- **Objetivos de aprendizaje**:
  - Utilizar estructuras condicionales para controlar el flujo de un programa.
  - Implementar bucles para repetir tareas de manera eficiente.
- **Referencias**:
  - Downey (2015): Capítulo 4 (**Condicionales y recursión**) y Capítulo 6 (**Iteración**).
  - Guttag (2016): Capítulo 2 (**Introducción a Python**) y Capítulo 3 (**Algunos programas simples**).
  - **Van Rossum (2015)**: Secciones sobre **Control de Flujo** en el **Tutorial de Python**.

Unidad 4: Funciones y modularización

- **Temas principales**:
  - Primitivas, procedimientos y funciones.
  - Funciones con y sin parámetros.
  - Propiedades del código: legibilidad y reutilización.
- **Objetivos de aprendizaje**:
  - Diseñar y utilizar funciones para modularizar programas.
  - Comprender la importancia de la legibilidad y la reutilización del código.
- **Referencias**:
  - Downey (2015): Capítulo 3 (**Funciones**) y Capítulo 5 (**Funciones productivas**).
  - Guttag (2016): Capítulo 4 (**Funciones**).
  - **Van Rossum (2015)**: Secciones sobre **Funciones** en el **Tutorial de Python**.
  - **Martin (2012)**: Capítulos sobre **Funciones limpias** y **Código legible** en **Código Limpio**.

Unidad 5: Prácticas de programación

- **Temas principales**:
  - Depuración (debugging) y verificación (testing).
  - Buenas prácticas de programación.
- **Objetivos de aprendizaje**:
  - Aprender a identificar y corregir errores en el código.

  - Implementar pruebas para verificar el correcto funcionamiento de los programas.
  - Adoptar buenas prácticas de programación, como la legibilidad y la organización del código.
- **Referencias**:
  - Downey (2015): Capítulo 1 (**¿Qué es la depuración (debugging)?**) y Capítulo A (**Depuración**).
  - Guttag (2016): Capítulo 6 (**Pruebas y depuración**).
  - **Martin (2012)**: Capítulos sobre **Depuración** y **Pruebas** en **Código Limpio**.

Unidad 6: Introducción a estructuras de datos básicas

- **Temas principales**:
  - Manejo de secuencias de datos: rangos, cadenas y listas.
  - Uso de estructuras secuenciales en la resolución de problemas.
  - Diferenciación de estructuras mutables e inmutables.
- **Objetivos de aprendizaje**:
  - Comprender el uso de listas y cadenas para almacenar y manipular datos.
  - Aplicar estructuras de datos en la implementación de algoritmos.
- **Referencias**:
  - Downey (2015): Capítulo 7 (**Cadenas**) y Capítulo 8 (**Listas**).
  - Guttag (2016): Capítulo 5 (**Estructuras de control**).
  - **Van Rossum (2015)**: Secciones sobre **Listas** y **Cadenas de Caracteres** en el **Tutorial de Python**.

Contenido Opcional - Unidad 7: Temas avanzados de programación

---

Tema 1: Tipos de datos computacionales de dos dimensiones (matrices y diccionarios)

- **Descripción**:
  - Uso de matrices y diccionarios para modelar e interpretar datos en problemas complejos.
  - Aplicaciones prácticas, como el manejo de datos tabulares o estructuras de datos anidadas.

- **Referencias**:
  - **Downey (2015)**:
    - Capítulo 10 (**Listas**): Introduce listas y estructuras de datos básicas, que son la base para entender matrices.
    - Capítulo 11 (**Diccionarios**): Explica el uso de diccionarios para almacenar y manipular datos.
  - **Guttag (2016)**:
    - Capítulo 5 (**Estructuras de control**): Incluye ejemplos de uso de listas y diccionarios.
    - Capítulo 8 (**Manejo de archivos**): Muestra cómo trabajar con datos estructurados.
  - **Van Rossum (2015)**:
    - Secciones sobre **Listas** y **Diccionarios** en el **Tutorial de Python**.
  - **Martin (2012)**:
    - Capítulos sobre **Diseño de código**: Ofrece consejos sobre cómo estructurar datos de manera eficiente.

Tema 2: Interpretación de documentación de módulos y funciones de terceros

- **Descripción**:
  - Cómo leer y utilizar documentación de módulos y funciones programadas por terceros.
  - Enfoque en el uso de módulos como "cajas negras" para resolver problemas.
  - Ejemplos prácticos: uso de módulos de Python para gráficos (matplotlib), lectura/escritura de archivos (os, csv), etc.
- **Referencias**:
  - **Downey (2015)**:
    - Capítulo 14 (**Archivos**): Introduce el manejo de archivos en Python.
    - Capítulo 4 (**Caso de estudio: diseño de interfaz**): Muestra cómo utilizar módulos externos.
  - **Guttag (2016)**:
    - Capítulo 8 (**Manejo de archivos**): Cubre el uso de módulos para trabajar con archivos.
    - Capítulo 9 (**Visualización de datos**): Introduce módulos para gráficos.
  - **Van Rossum (2015)**:
    - Secciones sobre **Módulos y paquetes** en el **Tutorial de Python**.
  - **Martin (2012)**:
    - Capítulos sobre **Diseño de interfaces**: Ofrece consejos sobre cómo interactuar con módulos externos.

Tema 3: Definición de tests de caja negra (Partición de Equivalencia)

- **Descripción**:
  - Técnica de Partición de Equivalencia para reducir la cantidad de casos de prueba sin perder cobertura.
  - Ejemplos prácticos de aplicación en programas simples.
- **Referencias**:
  - **Downey (2015)**:
    - Capítulo 11 (**Depuración**): Introduce conceptos básicos de testing.
  - **Guttag (2016)**:
    - Capítulo 6 (**Pruebas y depuración**): Explica técnicas de testing, incluyendo pruebas de caja negra.
  - **Martin (2012)**:
    - Capítulos sobre **Pruebas**: Ofrece una visión detallada de las técnicas de testing, incluyendo Partición de Equivalencia.

Tema 4: Definición de tests de caja blanca (Cobertura de ramas, condiciones, funciones)

- **Descripción**:
  - Técnicas de cobertura de ramas, condiciones y funciones para pruebas de caja blanca.
  - Ejemplos prácticos de implementación en Python.
- **Referencias**:
  - **Downey (2015)**:
    - Capítulo 11 (**Depuración**): Introduce conceptos básicos de pruebas de caja blanca.
  - **Guttag (2016)**:
    - Capítulo 6 (**Pruebas y depuración**): Cubre técnicas avanzadas de testing, incluyendo cobertura de ramas y condiciones.
  - **Martin (2012)**:
    - Capítulos sobre **Pruebas**: Profundiza en técnicas de cobertura de código y pruebas de caja blanca.

d. Metodología y evaluación

La materia de **Programación 1** se desarrollará mediante un enfoque **teórico-práctico**, combinando clases expositivas con ejercicios guiados, proyectos prácticos y actividades colaborativas. El objetivo es fomentar un aprendizaje activo y significativo, donde los estudiantes puedan aplicar los conceptos teóricos en situaciones reales de programación.

Modalidad de cursada

- **Horas de cursada**: La materia cuenta con un total de **8 horas semanales de trabajo presencial**.
- **Modalidad aula/taller**: Las clases se desarrollarán en un formato de **aula/taller**, donde se alternarán explicaciones teóricas con ejercicios prácticos. Los estudiantes trabajarán en computadora, utilizando **Python** como lenguaje de programación principal. Se incluirá también el uso de actividades desenchufadas para programación, depuración de código y escritura de diagramas y pseudocódigo.
- **Dinámica de las clases**:
    - **Inicio de la clase**: Se dedicará un espacio para revisar los temas abordados en la clase anterior, resolver dudas y responder preguntas.
    - **Desarrollo de la clase**: Se usarán distintas estrategias, a veces se presentarán nuevos conceptos teóricos, seguidos de ejercicios guiados y actividades prácticas y otras de desafíos cuyo abordaje requiera la asimilación de nuevos conceptos. Estas actividades se realizan de forma **individual** y **grupal**, fomentando la colaboración y el aprendizaje entre pares.
    - **Cierre de la clase**: Al finalizar cada sesión, se propiciará un espacio de **síntesis y reflexión** sobre los temas abordados, permitiendo a los estudiantes consolidar su aprendizaje y revisar sus apuntes.

Seguimiento del avance

- **Repositorio de trabajo**: Cada estudiante deberá contar con un **repositorio en línea** (por ejemplo, en **GitHub** o **Google Drive**) donde subirá de forma periódica el material desarrollado durante la cursada. Esto incluirá códigos, ejercicios resueltos, apuntes y proyectos.
- **Evaluación formativa**: La evaluación se concibe como un **proceso constructivo y continuo**. Los estudiantes recibirán retroalimentación constante sobre sus trabajos durante las clases presenciales en base al contenido que vayan elaborando en sus repositorios.

Herramientas y buenas prácticas

- **Verificación automática**: Las entregas de código, para fomentar la calidad del mismo, deberán pasar los casos de prueba propuestos. Se utilizarán herramientas de **verificación automática**, como:
    - **Casos de tests**: Propuestos por docentes y estudiantes, para validar el correcto funcionamiento de los programas.
    - **Linters**: Herramientas que analizan el código para asegurar el cumplimiento de **buenas prácticas de programación**, **convenciones de estilo** y la detección de errores sutiles.

- **Autorregulación del aprendizaje**: Se utilizarán **rúbricas** e **inventarios de conocimiento previo (KPSI)** para ayudar a los estudiantes a autorregular su aprendizaje, identificando fortalezas y áreas de mejora durante la cursada.

Evaluación y calificación

- **Calificación**: La calificación final se compondrá de las notas de dos evaluaciones parciales individuales en computadora y en papel donde las y los estudiantes deberán resolver problemas de programación similares a los trabajados durante las clases.
- **Proceso formativo**: La interacción entre estudiantes, la capacidad de enseñar y aprender entre pares, y la participación activa en las actividades serán consideradas como parte del **proceso evaluativo formativo** durante las clases.

e. Requisitos para la acreditación

Condición de regularidad:

- Asistencia a clase: mínimo de 70%
- Habrá dos exámenes: Examen 1 y Examen 2. El Examen 1 evalúa los contenidos hasta la unidad 3. El Examen 2 incluye todo el contenido del curso.
- El examen 1 no tendrá instancia de recuperación, sus conceptos serán evaluados en el integrador examen 2.
- El examen 2 tendrá instancia de recuperación.
- La calificación se calculará con la siguiente formula: Si en Examen 2 la calificación es mayor o igual a 5: max(Examen 1, Examen 2) * 30% + Examen 2 * 70. Si no la calificación será Examen 2.
- Para regularizar la materia se requiere un puntaje mayor a 4.

Condición de acreditación:

- Se promociona con calificación final de cursada de la materia mayor a 7.

f. Bibliografía Obligatoria

- DOWNEY, ALLEN (2002). Aprenda a pensar como un programador - con Python (Traducción). Recuperado de https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf
- VAN ROSSUM, GUIDO (2015) El tutorial de Python (Traducción).

UNIVERSIDAD PEDAGÓGICA NACIONAL

g. Bibliografía Complementaria

- MARTIN, ROBERT C. (2012) Código Limpio - Manual de estilo para el desarrollo ágil de software (Traducción).
- Guttag, J. V. (2016). Introduction to computation and programming using Python: With application to understanding data (2nd ed.). The MIT Press.
- Wachenchauzer et al. (2012) Aprendiendo a programar usando Python como herramienta.
- Python Software Foundation. (s.f.). *Documentación de Python (versión 3.13)*. Python.org. Disponible en https://docs.python.org/es/3.13/.
- Python Software Foundation. (s.f.). *Tutorial de Python (versión 3.13)*. Python.org. Disponible en https://docs.python.org/es/3.13/tutorial/